

FLIP FLOP DEVRELERİ

Devre çalıştığı sürece çıkış durumunu koruyan yapılara Flip-Flop denir. Flip-Floplar temel hafıza birimini oluştururlar. 1 bitlik bilgiyi saklayabilirler. Flip-Flop'larda bu bilgiyi ve bu bilginin tümleyenini gösteren (kendisi ve değili) iki çıkışı bulunur (Q ve Q'). Bir Flip-Flop'nin çıkışı dendiğinde bu çıkışın kendisidir (Q). Çıkışın değişmesi için girişin değişmesi ve tetikleme değişmesi gerekmektedir. Tetikleme değişmediği sürece çıkış konumunu korur. Flip-Flop'larda tetikleme clock pulse (CP) denilen kare dalga sinyal ile yapılır. Devrenin niteliğine göre kare dalganın yükselen kenarıyla ya da düşen kenarıyla tetikleme yapılabilir. Nadir olarak clock pulse'in pozitif düşey veya negatif düşey durumuyla da tetikleme yapılabilmektedir. Bu durum devrede özel olarak belirtilir.

D-Flip Flop

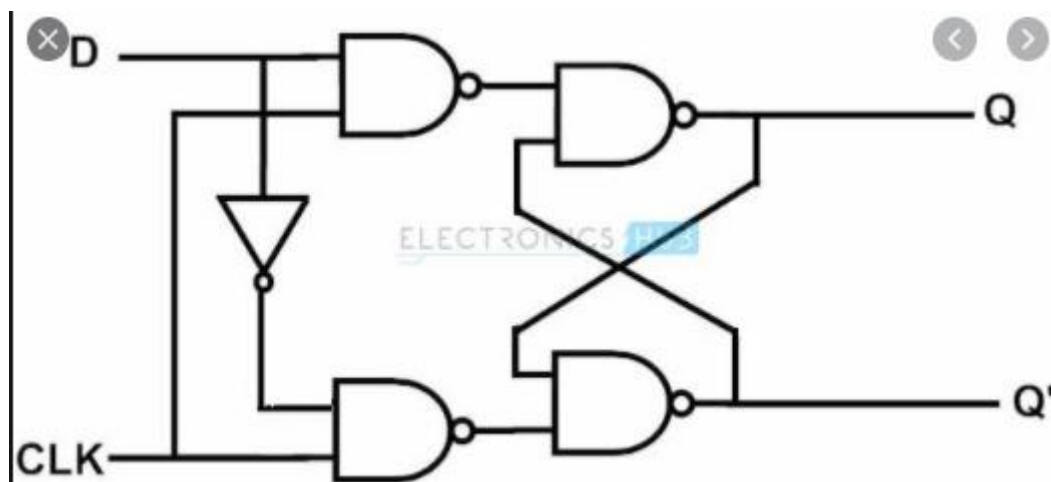
Girişin biri diğerinin tümleyeni olarak tek girişe yönlendirilmiştir. Data Flip-Flop D girişindeki bilgiyi her tetiklemeyle birlikte Q çıkışına aktarır.

D Flip Flop Truth Table

CL (Note 1)	D	R	S	Q	$\overline{Q}$
	0	0	0	0	1
	1	0	0	1	0
	x	0	0	Q	$\overline{Q}$
x	x	1	0	0	1
x	x	0	1	1	0
x	x	1	1	1	1

No Change

x = Don't Care Case



ÖRNEK

- 1) $y = (b \cdot c) + (a \cdot b)$ ifadesine ait devreyi bir donanım için verilog kodu ile yazınız. Bir testbench yazarak simülasyonu gerçekleştiriniz.
- 2) Diğer flip flopların devrelerine ait donanım kodlarını verilog dili kullanarak yazınız. Bir testbench yazarak simülasyonu gerçekleştiriniz.

IF

If (condition)

begin

....

end else

begin

....

end

örnek

```
reg f, g; // a new reg variable, g
always @(sel or a or b)
begin
    if (sel == 1)
        begin
            f = a;
            g = ~a;
        end
    else
        begin
            f = b;
            g = a & b;
        end
    end
end
```

örnek

```
reg f, g, h; // yes, an extra reg variable, h
always @(sel or sel_2 or a or b)
begin
    if (sel == 1)
    begin
        f = a;
        if (sel_2 == 1)
        begin
            h = ~b;
            g = ~a;
        end
    else
        begin
            g = a | b;
            h = a & b;
        end
    end
else
    begin
        if (sel_2 == 1)
        begin
            g = a & b;
            h = ~(a & b);
        end
    else
        begin
            h = ~(a | b);
            g = a | b;
        end
        f = b; // here's f!
    end
end
```

CASE

Yazım Kuralı

```
case (<expression>)
    case_item1 :    <single statement>
    case_item2,
    case_item3 :    <single statement>
    case_item4 :    begin
                        <multiple statements>
                    end
    default      : <statement>
endcase
```

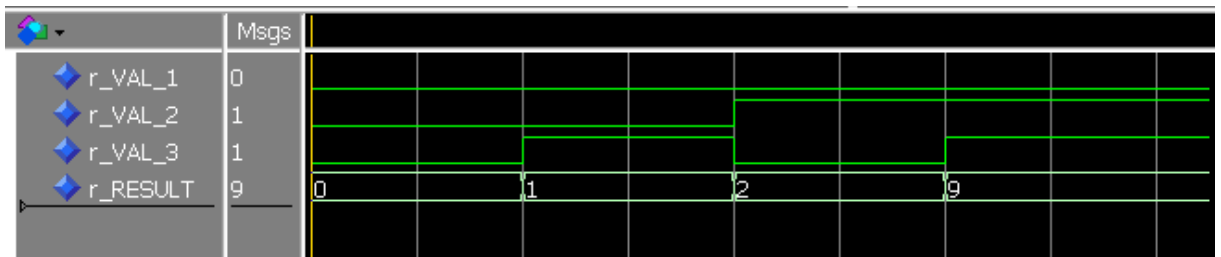
Örnek 1

```
always @ (a, b, c, sel) begin
    case(sel)
        2'b00 : out = a;
        2'b01 : out = b;
        2'b10 : out = c;
        default : out = 0;
    endcase
end
```

Örnek 2

```
case ({r_VAL_1, r_VAL_2, r_VAL_3})
    3'b000 : r_RESULT <= 0;
    3'b001 : r_RESULT <= 1;
    3'b010 : r_RESULT <= 2;
    default : r_RESULT <= 9;
endcase
```

Örnek 2 Simulasyonu



FOR LOOP

Yazım Kuralları

```
for (index = low_range; index < high_range; index = index + step)
for (index = high_range; index > low_range; index = index - step)
for (index = low_range; index <= high_range; index = index + step)
for (index = high_range; index >= low_range; index = index - step)
```

Örnek

```
for (i = 0; i <= 31; i = i + 1) begin
    s[i] = a[i] ^ b[i] ^ carry;
    carry = a[i] & b[i] | a[i] & carry |
            b[i] & carry;
end
```

Örnek

```
for (i = 6; i >= 0; i = i - 1)
  for (j = 0; j <= i; j = j + 1)
    if (value[j] > value[j+1]) begin
      temp = value[j+1];
      value[j+1] = value[j];
      value[j] = temp;
    end
  end
```

WHILE LOOP

Yazım Kuralı

```
while (condition)
  begin
    .....
  end
```

Örnek

```
always
  while (x < y)
    x = x + z;
```

Örnek

```
always
  begin @ (posedge clock)
    while (x < y)
      begin
        @ (posedge clock);
        x = x + z;
      end
  end;
end;
```

FOREVER LOOPS

Yazım Kuralı:

```
forever
begin
...

end
```

örnek

```
always
    forever
    begin
        @ (posedge clock);
        X = X + Z;
    end
```

ÖDEV:

- 4x1 MULTIPLEXER
- 2x4 DECODER